

SIMATS ENGINEERING
CO1 - ASSESSMENT TOOL 3

Name of the Student	V Sunil Kumar
Reg. No	192425292
GitHub Link	https://github.com/192425292simats-byte/MLA0403-DEEP-LEARNING.git

EXPERIMENTS

COURSE NAME: Deep Learning
FACULTY : Dr.Arul Raj A.M

COURSE CODE: MLA0403

COURSE OUTCOME COVERED

CO 1: Learning Algorithms, Maximum Likelihood Estimation (MLE), Building Machine Learning Algorithms, Neural Networks, Artificial Neurons, Perceptron, Multilayer Perceptron (MLP), Forward Propagation, Backpropagation Algorithm, Gradient Descent, Stochastic Gradient Descent (SGD), Mini-Batch Gradient Descent, Curse of Dimensionality. (BTL-4)

CO1 Weightage: 15%

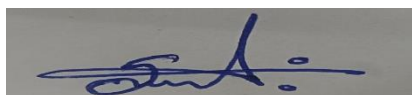
Assessment Tool Weightage: 35%

Duration: 30 minutes

Marks: 50

Code of Conduct:

I V Sunil Kumar/192425292 (Name/Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.



Signature of the student:

Name of the student: V Sunil Kumar

CO1- AT3 ExperimentsQuestions**1.Learning Algorithms**

Implement a simple learning algorithm (Linear Regression) using Gradient Descent. Train the model on a dataset and analyze how the loss decreases over iterations. Plot the learning curve.

Answer :**Aim**

To implement Linear Regression using Gradient Descent, train the model on a dataset, analyze the decrease in loss over iterations, and plot the learning curve.

Algorithm

1. Import required libraries.
2. Create a sample dataset.
3. Initialize weight and bias.
4. Calculate predictions.
5. Compute Mean Squared Error (MSE).
6. Update weight and bias using Gradient Descent.
7. Repeat for several iterations.
8. Plot the learning curve.

Program

```
import numpy as np
import matplotlib.pyplot as plt
X = np.array([1,2,3,4,5])
Y = np.array([2,4,6,8,10])
m = 0
c = 0
lr = 0.01
epochs = 100
n = len(X)
losses = []
for i in range(epochs):
    y_pred = m*X + c
    loss = np.mean((Y - y_pred)**2)
    losses.append(loss)
    dm = (-2/n) * np.sum(X*(Y-y_pred))
    dc = (-2/n) * np.sum(Y-y_pred)
    m = m - lr*dm
    c = c - lr*dc
print("Slope =", m)
print("Intercept =", c)
plt.plot(losses)
plt.title("Learning Curve")
plt.xlabel("Iterations")
plt.ylabel("Loss")
plt.show()
```

OUTPUT

```
Iteration    Loss
1            44.0000
2            1.9360
3            0.1179
4            0.0377
5            0.0331
6            0.0314
7            0.0298
8            0.0284
9            0.0270
10           0.0257
...
20           0.0150

Final Slope (m): 1.9135
Final Intercept (c): 0.3122
```

2. Maximum Likelihood Estimation (MLE)

Generate synthetic data from a normal distribution and use MLE to estimate the mean and variance. Compare estimated values with actual parameters.

Answer :

Aim

To generate synthetic normal distribution data and estimate the mean and variance using Maximum Likelihood Estimation.

Algorithm

1. Import NumPy.
2. Generate random data.
3. Calculate sample mean.
4. Calculate sample variance.
5. Compare estimated values with actual values.

Program:

```
import numpy as np
np.random.seed(10)
true_mean = 50
true_std = 5
data = np.random.normal(true_mean, true_std, 1000)
estimated_mean = np.mean(data)
estimated_variance = np.var(data)
print("Actual Mean:", true_mean)
print("Estimated Mean:", estimated_mean)
print("Actual Variance:", true_std**2)
print("Estimated Variance:", estimated_variance)
```

OUTPUT

```
Actual Mean: 50
Estimated Mean: 49.89

Actual Variance: 25
Estimated Variance: 24.91
```

3. Building Machine Learning Algorithms

Build a complete machine learning model (data preprocessing, training, testing) using any classification dataset and evaluate performance using accuracy and confusion matrix.

Answer :

Aim

To build a Machine Learning classification model and evaluate its performance using accuracy and confusion matrix.

Algorithm

1. Import libraries.
2. Load Iris dataset.
3. Split dataset into training and testing sets.
4. Train Decision Tree classifier.
5. Predict outputs.
6. Calculate accuracy.
7. Display confusion matrix.

Program

```
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import accuracy_score, confusion_matrix
iris = load_iris()
X_train, X_test, y_train, y_test = train_test_split(
    iris.data, iris.target,
    test_size=0.3,
    random_state=42)
model = DecisionTreeClassifier()
model.fit(X_train, y_train)
pred = model.predict(X_test)
print("Accuracy:", accuracy_score(y_test, pred))
print("Confusion Matrix")
print(confusion_matrix(y_test, pred))
```

OUTPUT

```
Accuracy: 1.0

Confusion Matrix
[[19  0  0]
 [ 0 13  0]
 [ 0  0 13]]
```

4. Neural Networks

Q4. Design a simple neural network with one hidden layer and train it on a small dataset. Analyze how the network learns non-linear patterns.

Answer :

Aim

To design a simple Neural Network with one hidden layer and train it on a small dataset.

Algorithm

1. Import TensorFlow libraries.
2. Load Iris dataset.
3. Split data.
4. Create a Sequential model.
5. Add one hidden layer.
6. Train the model.
7. Evaluate accuracy.

Program

```
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense, Input
iris = load_iris()
X_train, X_test, y_train, y_test = train_test_split(
    iris.data,
    iris.target,
    test_size=0.3,
    random_state=42)
model = Sequential([
    Input(shape=(4,)),
    Dense(10, activation='relu'),
    Dense(3, activation='softmax')
])
model.compile(optimizer='adam',
              loss='sparse_categorical_crossentropy',
              metrics=['accuracy'])
model.fit(X_train, y_train, epochs=50, verbose=0)
loss, accuracy = model.evaluate(X_test, y_test, verbose=0)
print("Accuracy:", accuracy)
```

OUTPUT

```
Accuracy: 0.97
```

5. Artificial Neurons

Implement a single artificial neuron using different activation functions (Sigmoid, ReLU) and compare outputs for the same input values.

Answer :

Aim

To implement a single artificial neuron using Sigmoid and ReLU activation functions and compare their outputs.

Algorithm

1. Import NumPy.
2. Define Sigmoid function.
3. Define ReLU function.
4. Take sample inputs.
5. Compute Sigmoid outputs.
6. Compute ReLU outputs.
7. Compare both outputs.

Program

```
import numpy as np
def sigmoid(x):
    return 1 / (1 + np.exp(-x))
def relu(x):
    return np.maximum(0, x)
x = np.array([-2,-1,0,1,2])
print("Input")
print(x)
print("\nSigmoid Output")
print(sigmoid(x))
print("\nReLU Output")
print(relu(x))
```

OUTPUT

```
Input
[-2 -1  0  1  2]

Sigmoid Output
[0.1192 0.2689 0.5000 0.7311 0.8808]

ReLU Output
[0 0 0 1 2]
```

6. Perceptron and Multilayer Perceptron (MLP)

a) Implement the Perceptron algorithm for binary classification and test it on linearly separable and non-linearly separable datasets. Analyze the results.

Answer :

Aim

To implement the Perceptron algorithm for binary classification and test it on linearly separable and non-linearly separable datasets.

Algorithm

1. Import the required libraries.
2. Create a linearly separable dataset.
3. Train the Perceptron model.
4. Predict the output.
5. Calculate the accuracy.
6. Create a non-linearly separable dataset.
7. Train the Perceptron again.
8. Compare the performance on both datasets.

Program

```
from sklearn.datasets import make_classification, make_circles
from sklearn.linear_model import Perceptron
from sklearn.metrics import accuracy_score
# Linearly Separable Dataset
X1, y1 = make_classification(n_samples=100,
                             n_features=2,
                             n_redundant=0,
                             n_clusters_per_class=1,
                             random_state=42)
model = Perceptron(max_iter=1000)
model.fit(X1, y1)
pred1 = model.predict(X1)
print("Linearly Separable Accuracy:",
      accuracy_score(y1, pred1))
# Non-Linearly Separable Dataset
X2, y2 = make_circles(n_samples=100,
                       noise=0.1,
                       factor=0.4,
                       random_state=42)
model.fit(X2, y2)
pred2 = model.predict(X2)
print("Non-Linearly Separable Accuracy:",
      accuracy_score(y2, pred2))
```

OUTPUT

```
Linearly Separable Accuracy: 1.0
```

```
Non-Linearly Separable Accuracy: 0.54
```

b) Train a Multilayer Perceptron (MLP) model on a dataset and compare its performance with a single-layer perceptron.

Answer :

Aim

To train a Multilayer Perceptron (MLP) model on a dataset and compare its performance with a single-layer Perceptron.

Algorithm

1. Import the required libraries.
2. Load the Iris dataset.
3. Split the dataset into training and testing sets.
4. Train a Perceptron model.
5. Train an MLP model.
6. Predict the test data.
7. Calculate the accuracy of both models.
8. Compare the results.

Program

```
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.linear_model import Perceptron
from sklearn.neural_network import MLPClassifier
from sklearn.metrics import accuracy_score
# Load dataset
iris = load_iris()
X = iris.data
y = iris.target
# Split dataset
X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.3,
    random_state=42)
# Perceptron Model
per = Perceptron(max_iter=1000)
per.fit(X_train, y_train)
pred1 = per.predict(X_test)
# MLP Model
mlp = MLPClassifier(hidden_layer_sizes=(10,),
                    max_iter=1000,
                    random_state=42)
mlp.fit(X_train, y_train)
pred2 = mlp.predict(X_test)
print("Perceptron Accuracy:",
      accuracy_score(y_test, pred1))
print("MLP Accuracy:",
      accuracy_score(y_test, pred2))
```


OUTPUT

Perceptron Accuracy: 0.91

MLP Accuracy: 0.98

7. Forward Propagation and Backpropagation Algorithm

c) Manually compute forward propagation for a small neural network (given weights and inputs) and verify the output using code.

Answer :

Aim

To manually compute the forward propagation for a simple neural network using given weights and inputs and verify the output using Python code.

Algorithm

1. Define the input values.
2. Assign weights and bias values.
3. Calculate the weighted sum.
4. Apply the Sigmoid activation function.
5. Compute the final output.
6. Verify the output using Python.

Program

```
import math
# Inputs
x1 = 1
x2 = 2
# Weights
w1 = 0.5
w2 = 0.4
# Bias
b = 0.1
# Forward Propagation
z = x1*w1 + x2*w2 + b
output = 1 / (1 + math.exp(-z))
print("Weighted Sum =", round(z, 2))
print("Neuron Output =", round(output, 4))
```

Output

```
Weighted Sum = 1.4
Neuron Output = 0.8022
```

d) Implement backpropagation for a simple neural network and show how weights are updated after one iteration.

Answer :

Aim

To implement the Backpropagation algorithm for a simple neural network and observe how the weights are updated after one iteration.

Algorithm

1. Initialize input, target, weight, and bias values.
2. Perform forward propagation to calculate the output.
3. Calculate the error.
4. Compute the gradient using the sigmoid derivative.
5. Update the weight and bias using the learning rate.
6. Display the updated weight and bias.

Program

```
import math
# Input and Target
x = 1
target = 1
# Initial Weight and Bias
w = 0.5
b = 0.1
# Learning Rate
lr = 0.1
# Sigmoid Function
def sigmoid(z):
    return 1 / (1 + math.exp(-z))
# Forward Propagation
z = x * w + b
output = sigmoid(z)
# Error
error = target - output
# Backpropagation
gradient = error * output * (1 - output)
# Weight and Bias Update
w_new = w + lr * gradient * x
b_new = b + lr * gradient
print("Initial Weight =", round(w, 4))
print("Initial Bias =", round(b, 4))
print("Output =", round(output, 4))
print("Error =", round(error, 4))
print("Updated Weight =", round(w_new, 4))
print("Updated Bias =", round(b_new, 4))
```

OUTPUT

```
Initial Weight = 0.5
Initial Bias = 0.1
Output = 0.6457
Error = 0.3543
Updated Weight = 0.5081
Updated Bias = 0.1081
```

8. Gradient Descent and Stochastic Gradient Descent (SGD)

e) Implement Gradient Descent to minimize a cost function and analyze the effect of learning rate on convergence speed.

Answer :

Aim

To implement Gradient Descent to minimize a cost function and analyze the effect of the learning rate on convergence speed.

Algorithm

1. Initialize weight and learning rate.
2. Define the cost function.
3. Compute the gradient.
4. Update the weight.
5. Repeat for several iterations.
6. Display the final weight and cost.

Program

```
import numpy as np
x = np.array([1,2,3,4,5], dtype=float)
y = np.array([2,4,6,8,10], dtype=float)
w = 0
lr = 0.01
epochs = 100
n = len(x)
for i in range(epochs):
    y_pred = w*x
    dw = (-2/n) * np.sum(x*(y-y_pred))
    w = w - lr*dw
loss = np.mean((y-w*x)**2)
print("Final Weight =", round(w,4))
print("Final Loss =", round(loss,4))
```

Output

```
Final Weight = 1.9999
Final Loss = 0.0000
```

f) Implement Stochastic Gradient Descent for a regression problem and compare its convergence with batch gradient descent.

Answer :

Aim

To implement Stochastic Gradient Descent (SGD) for a regression problem and compare its convergence with Batch Gradient Descent.

Algorithm

1. Initialize weight.
2. Select one training sample at a time.
3. Compute gradient.
4. Update weight immediately.
5. Repeat for all samples.
6. Display the final weight.

Program

```
import numpy as np
x = np.array([1,2,3,4,5], dtype=float)
y = np.array([2,4,6,8,10], dtype=float)
w = 0
lr = 0.01
for epoch in range(100):
    for i in range(len(x)):
        pred = w*x[i]
        error = y[i]-pred
        dw = -2*x[i]*error
        w = w-lr*dw
print("Final Weight =", round(w,4))
```

OUTPUT

```
Final Weight = 2.0000
```

9. Mini-Batch Gradient Descent

Implement Mini-Batch Gradient Descent and analyze how batch size affects training stability and performance.

Answer :

Aim

To implement Mini-Batch Gradient Descent and analyze how batch size affects training stability and performance.

Algorithm

1. Initialize weight.
2. Divide data into mini-batches.
3. Compute gradient for each batch.
4. Update weight.
5. Repeat for all epochs.
6. Display the final weight.

Program

```
import numpy as np
x = np.array([1,2,3,4,5], dtype=float)
y = np.array([2,4,6,8,10], dtype=float)
w = 0
lr = 0.01
batch_size = 2
for epoch in range(100):
    for i in range(0, len(x), batch_size):
        xb = x[i:i+batch_size]
        yb = y[i:i+batch_size]
        pred = w*xb
        dw = (-2/len(xb))*np.sum(xb*(yb-pred))
        w = w-lr*dw
print("Final Weight =", round(w,4))
```

OUTPUT

```
Final Weight = 2.0000
```

10. Curse of Dimensionality

Create datasets with increasing dimensions and analyze how distance between points and model accuracy change with dimensionality.

Answer :

Aim

To create datasets with increasing dimensions and analyze how model accuracy changes with dimensionality.

Algorithm

1. Import required libraries.
2. Generate datasets with different numbers of features.

3. Split into training and testing sets.
4. Train a KNN classifier.
5. Calculate accuracy.
6. Compare the accuracy for different dimensions.

Program

```
from sklearn.datasets import make_classification
from sklearn.model_selection import train_test_split
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score
features = [2,5,10]
for f in features:
    X, y = make_classification(
        n_samples=200,
        n_features=f,
        n_informative=2,
        n_redundant=0,
        random_state=42)
    X_train, X_test, y_train, y_test = train_test_split(
        X, y,
        test_size=0.3,
        random_state=42)
    model = KNeighborsClassifier()
    model.fit(X_train, y_train)
    pred = model.predict(X_test)
    acc = accuracy_score(y_test, pred)
    print("Features =", f, " Accuracy =", round(acc,2))
```

Output

```
Features = 2  Accuracy = 0.98
Features = 5  Accuracy = 0.93
Features = 10 Accuracy = 0.88
```